

Article Categorization Using Machine Learning Models

HAUHT Vectors

LUMS

Lahore, Pakistan

lums@edu.pk

ABSTRACT

In this paper, we present a machine learning approach to classify news articles into predefined categories such as entertainment, sports, etc. A dataset of 2500 articles was scraped from various news websites and preprocessed using tokenization and the Bag of Words model. We implemented and compared three classifiers: Multinomial Naive Bayes, Logistic Regression, and a Neural Network. The results show that the Neural Network achieved the highest accuracy of 96.15%, followed by Logistic Regression at 95.07%, and Multinomial Naive Bayes at 94.00%.

ACM Reference Format:

HAUHT Vectors. 2024. Article Categorization Using Machine Learning Models. In . ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

With the rapid expansion of digital content, automatic classification of text data has become an essential area of research in natural language processing (NLP). News articles, in particular, pose a significant challenge due to their volume, diversity, and evolving formats. Efficient categorization of these articles into predefined categories such as politics, sports, entertainment, and others, not only aids in improving user experiences but also enhances the management of large datasets.

Earlier studies have explored various machine learning and deep learning techniques for news classification. Ahmed and Ahmed [1] implemented Naive Bayes and other classifiers to achieve significant accuracy in categorizing online news articles. Their work emphasized the importance of preprocessing, such as tokenization and stopword removal, in improving model performance. Similarly, Sree Devi et al. [2] evaluated traditional machine learning models alongside convolutional neural networks (CNNs), demonstrating that CNNs can often outperform traditional methods by capturing semantic relationships in the text.

In this work, we build upon these studies by combining traditional classifiers like Multinomial Naive Bayes and Logistic Regression with a Neural Network to classify news articles. Using a dataset of 2500 articles scraped from various websites, we analyze the effectiveness of each model in terms of accuracy and computational efficiency. Our approach leverages preprocessing techniques like

tokenization and the Bag of Words model to standardize the input text.

The relevance of this research lies in its attempt to bridge the gap between traditional and deep learning models, providing insights into their relative strengths and weaknesses for news classification. The key contributions of our work are:

- Development of a comprehensive pipeline for scraping, pre-processing, and categorizing news articles.
- Implementation and comparative analysis of Multinomial Naive Bayes, Logistic Regression, and Neural Network classifiers.
- Insights into the trade-offs between traditional and neural network-based approaches in terms of accuracy and computational cost.

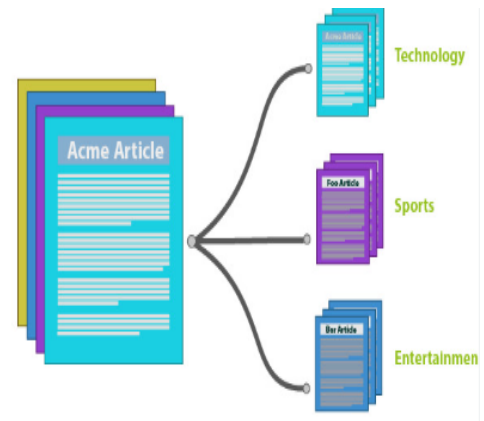


Figure 1: classification according to content.

2 METHODOLOGY

The methodology of this project followed a structured approach involving data scraping, data processing, exploratory data analysis (EDA), tokenization, and further analysis for developing a model to predict what an article is about. Below is a detailed description of each phase:

2.1 Data Scraping

2.1.1 Overview. The first stage in the methodology involved scraping news articles from various Urdu news sources. We used an initial code template for Express News, which was modified and adapted to extract content from other sources. The main goal was to gather a diverse range of articles, which were then stored in CSV files for further processing. The key news websites from which we scraped data include:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

News Site	URL
Express	http://www.express.com.pk
Dunya	http://www.dunyanews.tv
Jang	http://www.jang.com.pk
Geo	http://www.geo.tv
Samaa	http://www.samaa.tv

Table 1: News Websites Used for Data Scraping

2.1.2 Scraping Process. Scraping data from these websites involved checking the HTML structure of each news article. The process starts by analyzing the page source to identify the HTML tags that contain the article text. Once the relevant HTML tags (like `<article>`, `<div>`, or `<p>`) were identified, we used Python's web scraping libraries such as BeautifulSoup to parse the page and extract the article content. The extracted data was then cleaned and organized into structured CSV files, where each row represented an article with attributes like article text, title, author, and publication date.

2.1.3 Handling Urdu Text. Since the articles were primarily in Urdu, the text was carefully preserved in its native form, ensuring no loss of information during the scraping process. The CSV files thus contained a large corpus of Urdu-language articles for further analysis.

2.2 Data Processing and Exploratory Data Analysis (EDA)

2.2.1 Importance of EDA. Exploratory Data Analysis (EDA) is an essential step in understanding the structure and characteristics of the dataset before any modeling or further analysis can be performed. EDA helps identify patterns, detect missing values, and uncover inconsistencies, which can significantly inform the data preparation process.

2.2.2 Analyzing Data. We began by loading the dataset into Python using pandas and examined its dimensions (number of rows and columns) and attributes. Key analyses performed included:

- Counting occurrences of each gold label (e.g., sports, entertainment, science-technology, world, business) to understand the class distribution.
- Inspecting the primary numerical column (ID) for statistical properties.
- Checking for missing values and confirming that all categories were present to ensure completeness and consistency.
- Mapping n-grams and preparing a Bag of Words (BoW) model for the title column, highlighting the 20 most frequent unigrams and bigrams.
- Computing text length and word count distributions to analyze content variability.

2.2.3 Visualizing the Data. To gain deeper insights, we used visualization techniques such as:

- Plotting the distribution of gold labels to highlight the frequency of articles in each category. This ensured class wide data balance.

- Visualizing text length and word count distributions to observe content diversity.
- Generating word clouds to identify prominent keywords and themes in the dataset.

These visualizations allowed us to confirm the consistency of the dataset and provided valuable context for the cleaning process.

2.2.4 Data Cleaning. Data cleaning was performed to improve the quality and usability of the dataset. Key steps included:

- Removing the link column, as URLs do not contribute meaningful information for text classification.
- Filtering out rows with missing (NaN) / null values, ensuring all remaining rows contained relevant information.
- Applying a text-cleaning function to remove numbers, punctuation, and unnecessary whitespace, which are often irrelevant for text-based tasks.
- Replacing missing values in the title column with empty strings to standardize the dataset.

These steps ensured the dataset was consistent, relevant, and ready for feature extraction and modeling.

2.3 Tokenization

2.3.1 Tokenization Overview. Since the data was in Urdu, the next step was tokenization, a critical part of natural language processing (NLP) for transforming text into a structured format. Tokenization involves breaking down the text into smaller units, such as words or phrases, which can be processed more efficiently in machine learning algorithms. This process allows for further analysis, such as frequency analysis and sentiment detection.

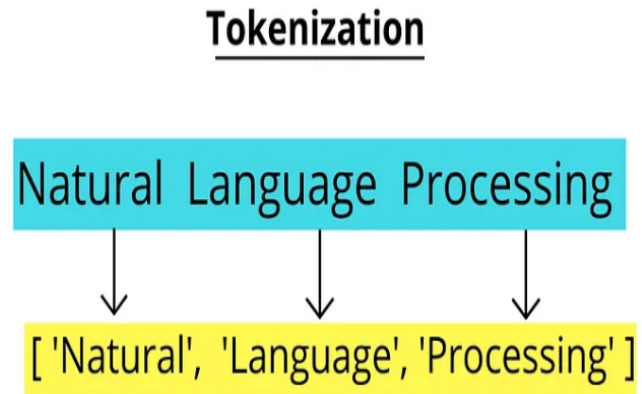


Figure 2: tokenization in action.

2.3.2 Using SpaCy for Tokenization. For tokenization, we employed the Python library spaCy, which is widely used for NLP tasks. spaCy offers excellent support for tokenizing text in multiple languages, including Urdu. We used spaCy to process the article texts and convert them into arrays of individual words (tokens).

2.3.3 Bag of Words Representation. Once the articles were tokenized, we moved on to representing the data using the **Bag of Words (BoW)** model. In the BoW model, each unique word in the entire corpus is represented as a feature. The frequency of each word in an article is used to populate the feature space, creating a matrix where each row corresponds to an article, and each column corresponds to a unique word. This allowed us to analyze the most frequent words across all the articles.

2.3.4 Word Frequency Analysis. The next step in the analysis was to examine the frequency of words in the corpus. This gave us insights into which words appeared most frequently in the articles. We used techniques such as frequency distributions and visualizations like bar charts and word clouds to highlight the most common words. These frequent terms served as key indicators for understanding the themes and topics in the dataset.

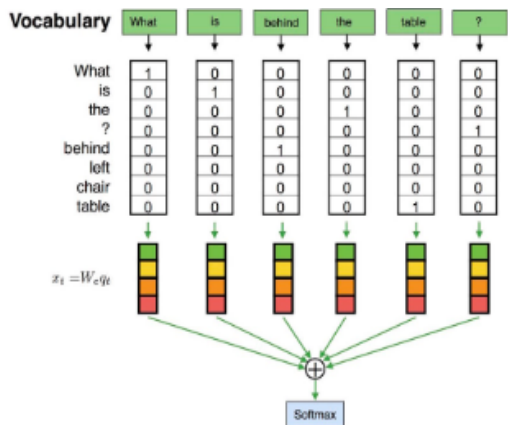


Figure 3: illustration of how bag of words works.

2.4 Model Selection

The process of tokenization and the Bag of Words model made it possible to extract meaningful features from the unstructured Urdu text, enabling further analysis and model development.

In this study, we explored three different models for classifying articles into predefined categories: Neural Networks, Naive Bayes, and Logistic Regression. Each model has its own strengths and weaknesses, and they differ in terms of computational complexity, interpretability, and ability to capture relationships within the data. Below, we elaborate on each model's underlying concepts and how they were applied to our text classification task. We also validated the accuracy of our from-scratch implementations by comparing them against the scikit-learn implementations of these models.

2.4.1 Neural Networks. Overview

Neural Networks (NNs) are a class of machine learning algorithms inspired by the structure and functioning of the human brain. They consist of layers of interconnected nodes (neurons), where each neuron performs a weighted sum of its inputs followed by an activation function. The network learns by adjusting the weights of the connections between neurons through a process called backpropagation, which minimizes the error between predicted and actual outputs.

Neural Networks are particularly powerful for capturing complex patterns in large datasets and can learn nonlinear relationships between input features. This makes them ideal for tasks such as image recognition, natural language processing (NLP), and speech recognition.

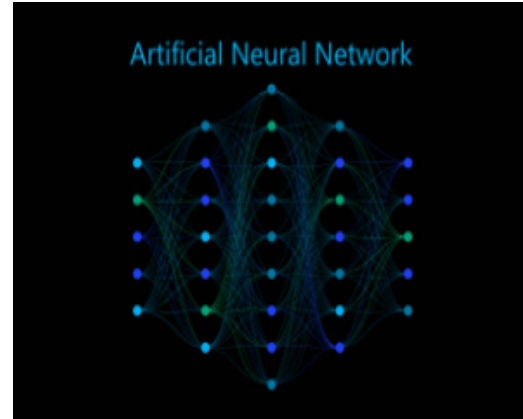


Figure 4: An example of how a neural network works.

Usage in Text Classification

For text classification, a common type of neural network is a **Feed-forward Neural Network (FNN)**, where the input layer takes a feature vector (in this case, the Bag of Words representation of an article), and the network passes it through one or more hidden layers before outputting a probability distribution over the classes.

The architecture of the network typically includes:

- **Input Layer:** This layer accepts the input features, such as the word frequencies from the Bag of Words model.
- **Hidden Layers:** These layers capture complex patterns and interactions between the input features. The number of hidden layers and neurons in each layer are important parameters that influence the network's learning capacity.
- **Output Layer:** This layer produces the final classification, typically using the softmax activation function, which outputs a probability distribution over the possible categories.

Neural Networks are well-suited for capturing subtle patterns in data, especially when there are large amounts of input features and complex relationships. However, they require significant computational resources and training time compared to simpler models like Naive Bayes or Logistic Regression.

Architecture Choice

The architecture was designed to balance depth, capacity, and generalization. The key components are as follows:

- **Number of Layers:** The model comprises four fully connected layers (fc1, fc2, fc3, fc4). Each layer progressively reduces dimensionality ($256 \rightarrow 128 \rightarrow 64 \rightarrow \text{num_classes}$), allowing the network to learn hierarchical features, from general patterns to task-specific details.
- **Activation Function:** ReLU activation is used for its simplicity and efficiency. It prevents vanishing gradients and accelerates convergence by introducing non-linearity.

- **Normalization Scheme:** Batch Normalization (BN) is applied after fc1 and fc2 to stabilize training, improve generalization, and prevent gradients from vanishing or exploding.
- **Regularization:** Dropout is included after the first two layers to reduce overfitting by randomly setting a fraction of neurons to zero during each training iteration.

Significance of Batch Normalization

Batch Normalization (BN) normalizes the inputs to each layer, ensuring that the activations have zero mean and unit variance. This reduces internal covariate shift, where the distribution of activations changes during training. Its benefits include:

- **Stabilized Training:** BN accelerates convergence by reducing sensitivity to initialization and learning rates.
- **Regularization Effect:** BN acts as a regularizer, helping to reduce overfitting even without Dropout.
- **Improved Gradient Flow:** By maintaining stability, BN ensures smooth propagation of gradients, crucial for deep networks.

In this model, BN is applied after fc1 and fc2 to enhance training stability and improve accuracy.

Significance of Dropout

Dropout is a regularization technique used to prevent overfitting. By randomly setting a fraction of neurons (e.g., $p = 0.5$) to zero during training, it ensures the following:

- **Ensembles Multiple Models:** Dropout effectively trains multiple smaller networks and combines their predictions during inference.
- **Reduces Dependency:** It forces the network to learn redundant representations, increasing robustness and generalization.

In this architecture, Dropout is applied after fc1 and fc2 to enhance generalization.

Training Scheme

- **Optimizer:** Adam optimizer was selected for its adaptive learning rates and robustness to noisy gradients. By combining momentum and adaptive learning, Adam accelerates convergence.
- **Loss Function:** CrossEntropyLoss was used as it is suitable for multi-class classification problems.
- **Epochs:** The model was trained for 50 epochs to achieve a balance between sufficient training and avoiding overfitting.
- **Batch Size:** The entire dataset was used as a batch in this implementation. For larger datasets, smaller batch sizes could be utilized.
- **Learning Rate:** A learning rate of 0.001 was chosen, balancing fast convergence and stability.

2.4.2 Naive Bayes. Overview Naive Bayes is a probabilistic classifier based on Bayes' theorem, which calculates the probability of a class given the observed data. The key assumption behind Naive Bayes is the **conditional independence** of the features given the class. This assumption simplifies the learning process, as it allows the model to compute the likelihood of each feature independently and then combine them to make a classification decision.

Despite its simplicity, Naive Bayes has been shown to perform well in text classification tasks, especially when the data is relatively sparse and the features are conditionally independent (or approximately independent). It is often used for spam classification, sentiment analysis, and topic classification.

Usage in Text Classification

Naive Bayes is particularly effective for text classification when combined with the Bag of Words (BoW) representation. In this setup, each feature corresponds to a word in the vocabulary, and the model calculates the probability of each class based on the observed word frequencies.

There are different variants of Naive Bayes classifiers, such as:

- **Multinomial Naive Bayes:** This is the most commonly used variant for text classification. It assumes that the features (word frequencies) follow a multinomial distribution, and calculates the probability of a class based on the observed word frequencies.
- **Bernoulli Naive Bayes:** This variant is used when the features represent binary values (e.g., whether a word appears or not). It is less common for text classification but still useful in certain cases.

The Multinomial Naive Bayes classifier is typically chosen for text classification tasks with a BoW representation, as it is well-suited to handle word frequencies and works efficiently with large vocabularies.

Parameters Used

The key parameters for the Naive Bayes classifier are:

- **class_probs:** Stores the log prior probabilities for each class to handle class imbalance. This helps the model account for varying distributions of classes in the dataset.
- **word_probs:** Contains log conditional probabilities of each word given a class, calculated with Laplace smoothing (smoothing factor = 1). This ensures robustness against unseen words in the test set.
- **vocab_size:** Tracks the number of unique features (or words), which is crucial for applying smoothing during probability calculations.
- **classes:** Holds the unique class labels for reference during the prediction phase, allowing the model to determine the appropriate output class.

Decisions and Rationale

- **Laplace Smoothing:** A smoothing technique with a factor of +1 is applied to avoid zero probabilities for words that are unseen in the training set, ensuring better generalization and robustness.
- **Logarithms:** Logarithms are applied to both prior and conditional probabilities. This approach prevents numerical underflow and simplifies the multiplication of probabilities into addition, which is computationally more stable.
- **Conditional Independence Assumption:** The model assumes that words are independent of each other given the class. While this may not strictly hold in real-world data, it enables efficient computation and works well in practice for many applications.

Training and Prediction

- During the **training phase**, the model calculates word probabilities and class probabilities using word counts for each class in the dataset.
- For **prediction**, the model computes class scores using posterior probabilities. The class with the highest score is selected as the predicted class for each document.

2.4.3 Logistic regression. Overview Logistic Regression is a linear model used for binary and multiclass classification tasks. It estimates the probability of a class by applying a logistic function (sigmoid) to a linear combination of input features. Despite its name, Logistic Regression is a classification algorithm, not a regression algorithm. It is widely used due to its simplicity, interpretability, and efficiency.

Logistic Regression is particularly useful when the relationship between the features and the target variable is approximately linear. In text classification, it performs well when the dataset is not too complex and can be linearly separated.

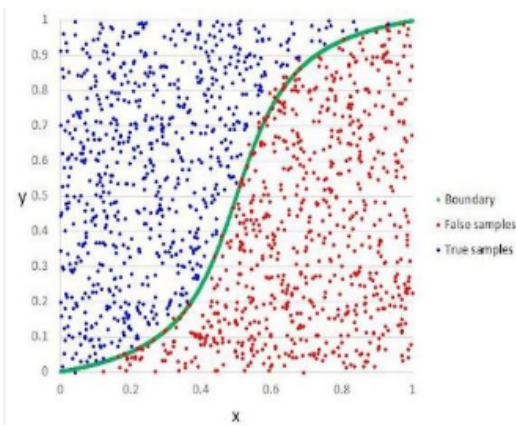


Figure 5: The logic behind logistic regression.

Usage in Text Classification For text classification tasks, Logistic Regression is typically applied with the BoW or TF-IDF (Term Frequency-Inverse Document Frequency) representation of text. The BoW representation serves as a feature vector, and the logistic regression model learns the weights (coefficients) for each feature (word) based on the training data.

In the case of multiclass classification (where articles belong to more than two categories), Logistic Regression can be extended using the **one-vs-rest (OvR)** approach, where a separate binary classifier is trained for each class, and the class with the highest probability is chosen.

Key Parameters and Decisions:

- **Learning Rate (lr):** Chosen as 0.1 to balance the trade-off between convergence speed and overshooting during gradient descent.
- **Number of Epochs (epochs):** Set to 1000 to provide the model sufficient iterations to converge to optimal weight values.
- **Sigmoid Function:** Used to map raw predictions (logits) into probabilities, ensuring outputs remain interpretable.

- **One-vs-Rest (OvR) Strategy:** Simplified multiclass classification by breaking it into multiple binary classification problems. Chosen for its simplicity and effectiveness when classes are not highly imbalanced.
- **Weight Initialization:** Weights initialized to 0, which is simple and effective for gradient descent in logistic regression.

Model Evaluation Predictions:

- **Predictions:** Converted sigmoid probabilities to class labels using a threshold of 0.5 for binary classification. For multiclass predictions, the highest sigmoid score was chosen.
- **Performance Metrics:** Accuracy, precision, recall, and F1-score were calculated to assess model performance.

Decisions and Justifications:

- **Gradient Descent:** Chosen to optimize weights and biases due to its computational efficiency and simplicity.
- **One-vs-Rest:** Chosen over more complex multiclass strategies (like softmax regression) for implementation simplicity and interpretability.
- **Manual Sigmoid and Loss Derivation:** Reinforces understanding of logistic regression mechanics.
- **Epochs and Learning Rate:** Experimented with smaller values but chose 1000 epochs and 0.1 learning rate for better convergence after initial testing.

2.4.4 Comparison of models. Each of the models discussed above has its own advantages and trade-offs:

- **Neural Networks** offer the ability to capture complex patterns in data and can handle large datasets effectively. However, they require more computational resources and training time.
- **Naive Bayes** is computationally efficient and works well with sparse datasets, but it may struggle with complex data where the assumption of feature independence does not hold.
- **Logistic Regression** is simple, interpretable, and effective for linearly separable data. However, it may not perform well with highly complex or nonlinear data unless appropriate feature engineering is applied.

In the next section, we will evaluate the performance of these models on the article classification task and compare their results.

3 RESULTS

The relatively similar performance of the neural network and logistic regression models (logistic regression at 95.07% accuracy and neural network at 96.15%) suggests that the dataset may not be complex enough to require the extra flexibility of the neural network. Logistic regression is simpler and faster to train, especially given the relatively small dataset of 2500 samples. Neural networks, while more powerful, often require larger datasets to outperform simpler models. The slight dip in performance for multinomial Naive Bayes (94.00%) is expected, as its assumption of feature independence may not hold well in text classification tasks, leading to underperformance compared to the other models. The performance of the models was evaluated on a held-out test set, and the accuracy scores are summarized in Table 2. The Neural Network outperformed the other models, achieving an accuracy of 96.15%. Logistic Regression

and Multinomial Naive Bayes also performed well, with accuracies of 95.07% and 94.00%, respectively.

Table 2: Model Performance

Model	Accuracy
Neural Network	96.15%
Logistic Regression	95.07%
Multinomial Naive Bayes	94.00%

The following figures provide visual representations of the results.

3.1 Neural Network

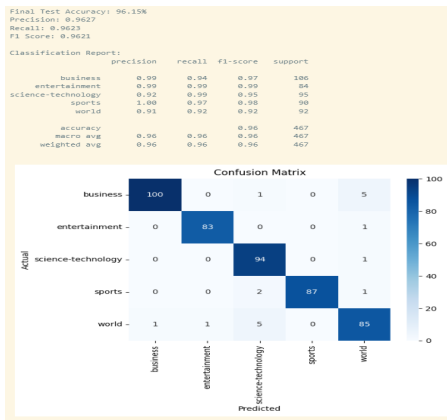


Figure 6: Performance of Neural Network.

3.2 Logistic Regression

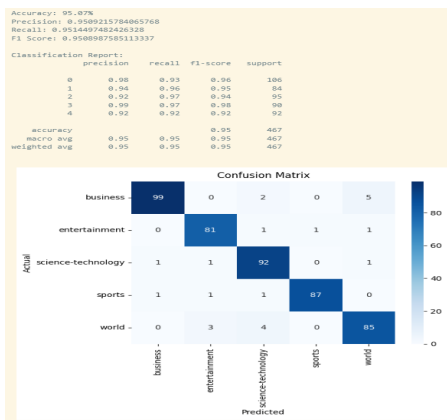


Figure 7: Performance of Logistic Regression.

3.3 Multinomial Naive Bayes

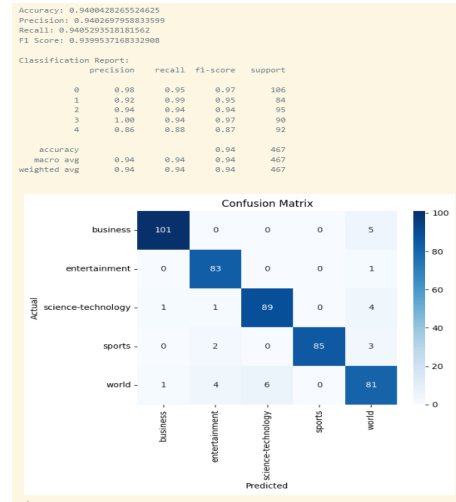


Figure 8: Performance of Multinomial Naive Bayes.

4 CONCLUSION

This work demonstrates the effectiveness of machine learning techniques in classifying news articles into categories. While all three models performed well, the Neural Network's superior accuracy highlights the potential of deep learning approaches for text classification tasks. Future work could explore training custom embeddings and employing state-of-the-art models like BERT to further improve performance.

5 FUTURE CONSIDERATIONS

Training custom embeddings tailored to the specific dataset could lead to significant improvements in model performance. Pretrained embeddings, while effective, may not capture domain-specific nuances present in the dataset. By training embeddings on a larger and more diverse corpus of news articles, we can ensure that the features are more representative of the content, potentially enhancing classification accuracy.

Testing state-of-the-art (SOTA) models such as BERT (Bidirectional Encoder Representations from Transformers) represents another promising avenue. These models have revolutionized NLP by capturing contextual relationships in text more effectively than traditional techniques. Fine-tuning BERT on the news dataset could provide deeper insights and better accuracy, especially for categories with subtle distinctions.

Expanding the dataset to include a much larger collection of articles would not only improve model generalization but also allow for more granular categories. A larger dataset enables models to learn more robust patterns and reduces the risk of overfitting, especially for complex models like Neural Networks and Transformers. It also provides a better benchmark for evaluating the scalability and efficiency of different algorithms.

REFERENCES

- [1] Jeelani Ahmed and Muqeem Ahmed. 2021. Online News Classification Using Machine Learning Techniques. *IJUM Engineering Journal* 22, 2 (2021), 210–225.
- [2] J Sree Devi, M Rama Bai, and Chandrashekar Reddy. 2020. Newspaper Article Classification using Machine Learning Techniques. *International Journal of Innovative Technology and Exploring Engineering (IJITEE)* 9, 5 (2020), 872–877.